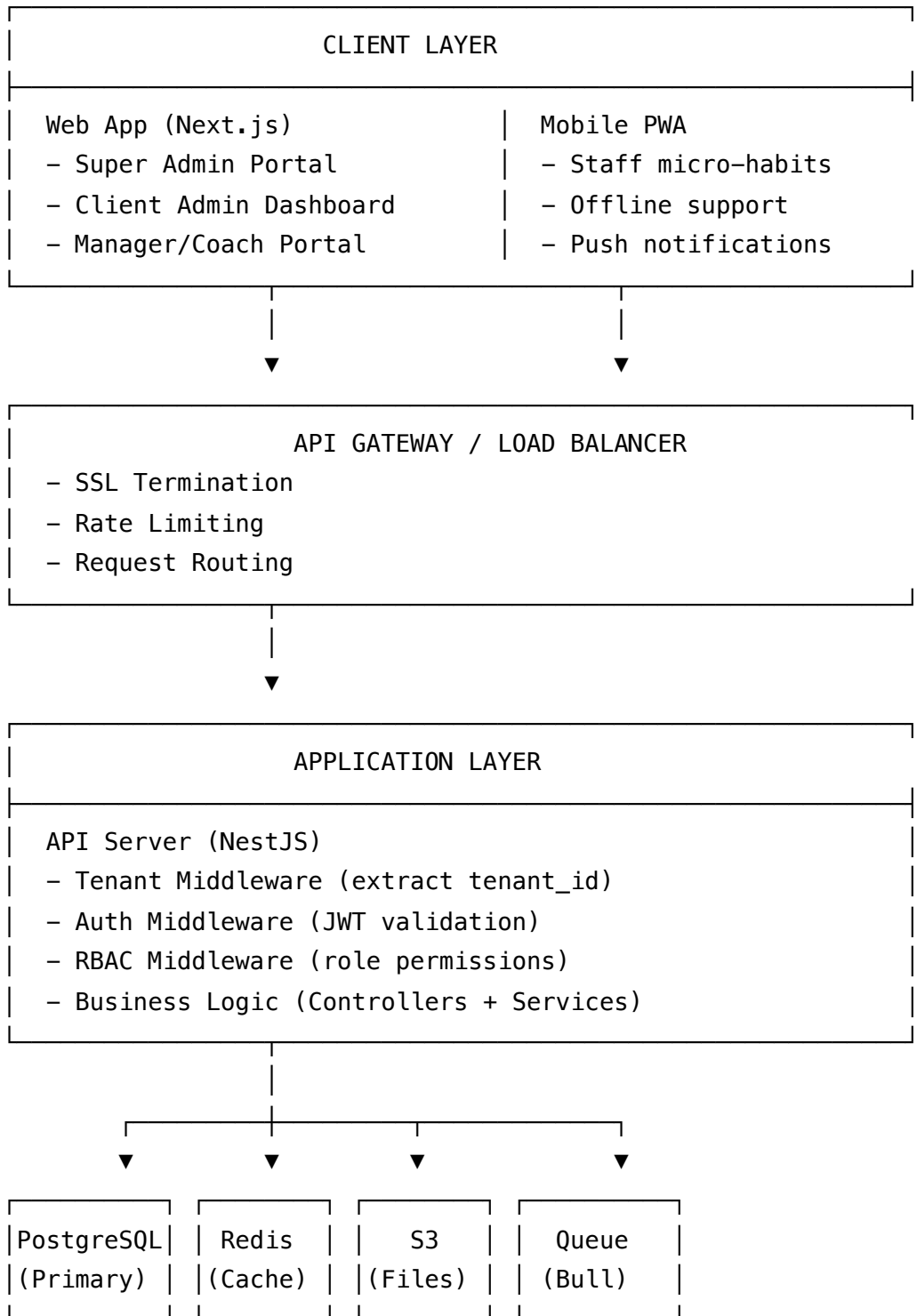


E2E Service System - System Design

1. High-Level Architecture



2. Technology Stack

Frontend

- **Framework:** Next.js 14+ (React 18+)
- **Language:** TypeScript
- **Styling:** Tailwind CSS
- **State:** Zustand or Redux Toolkit
- **API Client:** TanStack Query + Axios
- **PWA:** next-pwa plugin
- **Charts:** Recharts or Chart.js

Backend

- **Framework:** NestJS (Node.js)
- **Language:** TypeScript
- **ORM:** Prisma
- **Validation:** class-validator
- **Auth:** JWT + Passport
- **Queue:** Bull (Redis-based)
- **File Upload:** Multer + AWS SDK

Database

- **Primary:** PostgreSQL 15+
- **Cache:** Redis 7+
- **Search:** PostgreSQL Full-Text Search

Infrastructure

- **Containers:** Docker
- **Orchestration:** Kubernetes (optional)
- **IaC:** Terraform
- **Storage:** AWS S3 or GCS
- **CDN:** CloudFront or Cloudflare

Monitoring

- **Errors:** Sentry
- **Logs:** Winston + CloudWatch
- **Metrics:** Prometheus + Grafana
- **APM:** OpenTelemetry

3. API Design

RESTful Endpoints

Tenant Management (Super Admin)

```
POST    /api/admin/tenants
GET     /api/admin/tenants
GET     /api/admin/tenants/:id
PATCH  /api/admin/tenants/:id
DELETE  /api/admin/tenants/:id
```

Organization (Client Admin)

```
GET     /api/organizations/:id
PATCH  /api/organizations/:id
POST    /api/organizations/:id/targets
```

Branches

```
GET     /api/branches
POST    /api/branches
GET     /api/branches/:id
PATCH  /api/branches/:id
DELETE  /api/branches/:id
```

Users

GET /api/users
POST /api/users
POST /api/users/import (CSV)
GET /api/users/:id
PATCH /api/users/:id
DELETE /api/users/:id

Programs

GET /api/programs
POST /api/programs
GET /api/programs/:id
PATCH /api/programs/:id

Sessions

GET /api/sessions
POST /api/sessions
GET /api/sessions/:id
PATCH /api/sessions/:id
POST /api/sessions/:id/attendance

Roleplays

GET /api/roleplays
POST /api/roleplays
GET /api/roleplays/:id
POST /api/roleplays/:id/score

Micro-Habits

GET /api/habits
POST /api/habits
GET /api/habits/my
POST /api/habits/:id/complete
GET /api/habits/:id/streak

Coaching Logs

GET /api/coaching-logs
POST /api/coaching-logs
PATCH /api/coaching-logs/:id/approve
GET /api/coaching-logs/export

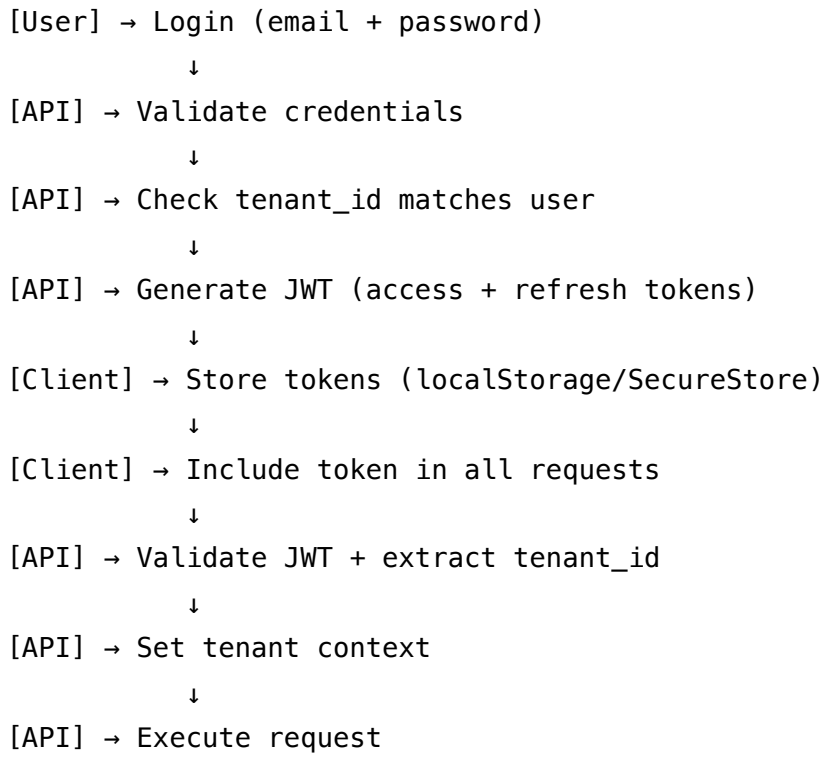
Metrics

GET /api/metrics
POST /api/metrics
POST /api/metrics/import (CSV)
GET /api/metrics/csat-trend
GET /api/metrics/complaints-trend
GET /api/metrics/branch-variance

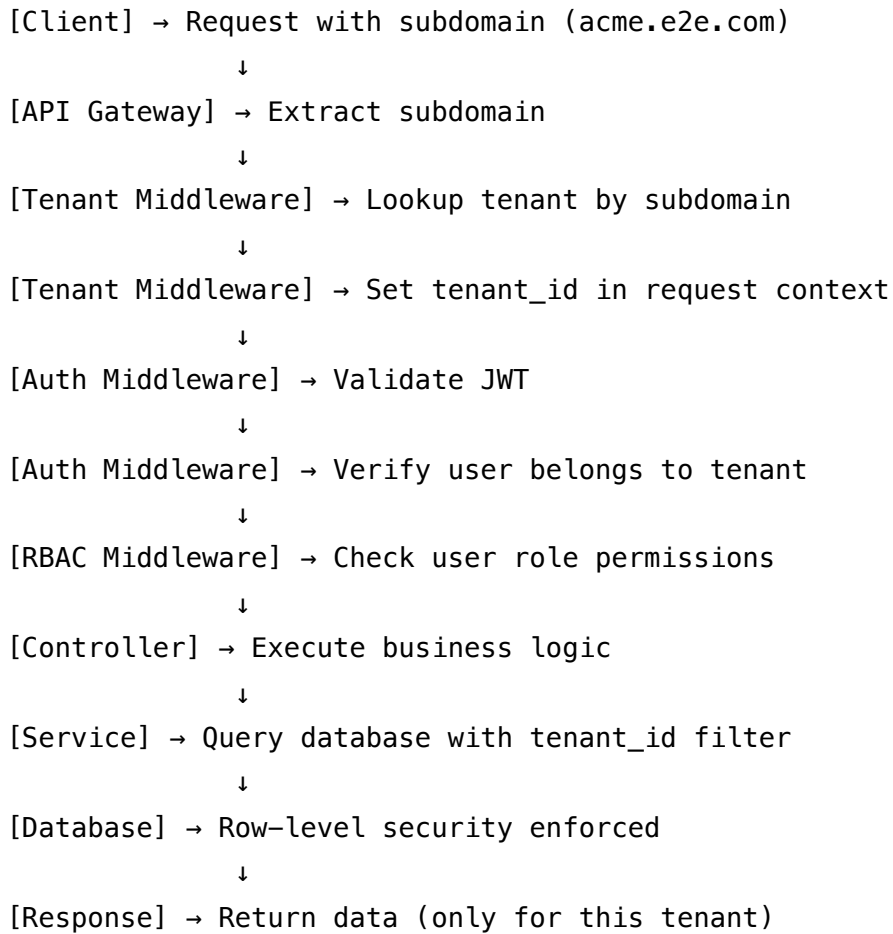
Reports

GET /api/reports/executive-scorecard (PDF)
GET /api/reports/branch-performance
POST /api/reports/schedule

4. Authentication Flow



5. Multi-Tenant Request Flow



6. Database Design Patterns

Tenant Isolation

```
// Every query includes tenant_id filter
async findUsers(tenantId: string) {
  return this.prisma.user.findMany({
    where: { tenantId },
  });
}

// Or use Prisma middleware
prisma.$use(async (params, next) => {
  if (params.model && params.action === 'findMany') {
    params.args.where = {
      ...params.args.where,
      tenantId: getCurrentTenantId(),
    };
  }
  return next(params);
});
```


Soft Deletes

```
// Don't actually delete, just mark as deleted
async softDelete(id: string) {
  return this.prisma.user.update({
    where: { id },
    data: { deletedAt: new Date() },
  });
}

// Filter out deleted records
async findAll(tenantId: string) {
  return this.prisma.user.findMany({
    where: {
      tenantId,
      deletedAt: null,
    },
  });
}
```

7. Caching Strategy

Redis Cache Layers

Session Cache

```
// Store JWT sessions
await redis.set(`session:${userId}`, JSON.stringify(session), 'EX', 86400);
```

Query Cache

```
// Cache expensive queries (5 min TTL)
const cacheKey = `tenant:${tenantId}:metrics:csat`;
const cached = await redis.get(cacheKey);
if (cached) return JSON.parse(cached);

const data = await this.calculateCSAT(tenantId);
await redis.set(cacheKey, JSON.stringify(data), 'EX', 300);
return data;
```

Rate Limiting

```
// Limit API requests per tenant
const key = `ratelimit:${tenantId}:${endpoint}`;
const count = await redis.incr(key);
if (count === 1) await redis.expire(key, 60);
if (count > 100) throw new TooManyRequestsException();
```

8. File Storage Architecture

S3 Bucket Structure

```
e2e-files/
├── tenant-1/
│   ├── sessions/
│   │   └── 2024-01-15-slides.pdf
│   ├── roleplays/
│   │   └── 2024-01-15-video.mp4
│   └── coaching/
│       └── 2024-01-15-photo.jpg
├── tenant-2/
│   └── ...
└── templates/ (global)
    └── 90-day-pilot.pdf
```

Upload Flow

```
async uploadFile(tenantId: string, file: Express.Multer.File) {
  const key = `${tenantId}/sessions/${Date.now()}-${file.originalname}`;

  await s3.upload({
    Bucket: 'e2e-files',
    Key: key,
    Body: file.buffer,
    ContentType: file.mimetype,
  }).promise();

  return `https://e2e-files.s3.amazonaws.com/${key}`;
}
```

9. Background Jobs

Job Types

Daily Jobs (Cron)

```
@Cron('0 0 * * *') // Midnight daily
async calculateDailyMetrics() {
  const tenants = await this.tenantService.findAll();

  for (const tenant of tenants) {
    await this.metricsService.calculateBranchVariance(tenant.id);
    await this.alertService.checkThresholds(tenant.id);
  }
}
```

Weekly Jobs

```
@Cron('0 9 * * 1') // Monday 9am
async sendWeeklyReports() {
  const tenants = await this.tenantService.findAll();

  for (const tenant of tenants) {
    const report = await this.reportService.generateWeekly(tenant.id);
    await this.emailService.send(tenant.adminEmail, 'Weekly Report', report);
  }
}
```

Queue Jobs (Bull)

```
// Add job to queue
await this.emailQueue.add('send', {
  to: 'user@example.com',
  subject: 'Welcome',
  body: 'Hello!',
});

// Process job
@Process('send')
async handleSendEmail(job: Job) {
  const { to, subject, body } = job.data;
  await this.emailService.send(to, subject, body);
}
```

10. Notification System

Channels

- Email (SendGrid or AWS SES)
- SMS (Twilio or Africa's Talking)
- WhatsApp (Twilio)
- Push (Firebase Cloud Messaging)
- In-App (WebSocket)

Notification Flow

```
async sendNotification(userId: string, type: string, message: string) {
  const user = await this.userService.findOne(userId);
  const tenant = await this.tenantService.findOne(user.tenantId);

  // Check tenant notification settings
  if (tenant.settings.emailEnabled) {
    await this.emailService.send(user.email, type, message);
  }

  if (tenant.settings.smsEnabled && user.phone) {
    await this.smsService.send(user.phone, message);
  }

  // Always send in-app notification
  await this.websocketGateway.sendToUser(userId, { type, message });
}
```

11. Security Architecture

JWT Structure

```
{
  "sub": "user-id",
  "email": "user@example.com",
  "role": "Manager",
  "tenantId": "tenant-id",
  "orgId": "org-id",
  "branchId": "branch-id",
  "iat": 1234567890,
  "exp": 1234567890
}
```

RBAC Implementation

```
@Injectable()
export class RolesGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const requiredRoles = this.reflector.get<string[]>('roles', context.getHandler());
    if (!requiredRoles) return true;

    const request = context.switchToHttp().getRequest();
    const user = request.user;

    // Check role
    if (!requiredRoles.includes(user.role)) {
      throw new ForbiddenException();
    }

    // Check tenant isolation
    if (request.params.tenantId && request.params.tenantId !== user.tenantId) {
      throw new ForbiddenException();
    }

    return true;
  }
}
```

12. Performance Optimization

Database Indexes

```
-- Composite indexes for common queries
CREATE INDEX idx_users_tenant_email ON users(tenant_id, email);
CREATE INDEX idx_sessions_tenant_date ON sessions(tenant_id, scheduled_at);
CREATE INDEX idx_metrics_tenant_type_date ON metrics(tenant_id, type, date);
```

Query Optimization

```
// Use select to fetch only needed fields
async findUsers(tenantId: string) {
  return this.prisma.user.findMany({
    where: { tenantId },
    select: {
      id: true,
      email: true,
      firstName: true,
      lastName: true,
      role: true,
    },
  });
}

// Use pagination
async findSessions(tenantId: string, page: number, limit: number) {
  return this.prisma.session.findMany({
    where: { tenantId },
    skip: (page - 1) * limit,
    take: limit,
    orderBy: { scheduledAt: 'desc' },
  });
}
```

Connection Pooling

```
// Prisma connection pool
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  connection_limit = 20
}
```

13. Monitoring & Observability

Metrics to Track

- API response time (p50, p95, p99)

- Error rate (per endpoint, per tenant)
- Database query time
- Cache hit rate
- Active users per tenant
- API requests per tenant

Logging

```
// Structured logging with Winston
logger.info('User logged in', {
  userId: user.id,
  tenantId: user.tenantId,
  role: user.role,
  ip: request.ip,
});
```

Error Tracking

```
// Sentry integration
Sentry.captureException(error, {
  tags: {
    tenantId: user.tenantId,
    userId: user.id,
  },
});
```

14. Scalability Strategy

Horizontal Scaling

- Deploy multiple API servers
- Load balancer distributes requests
- Kubernetes auto-scaling based on CPU/memory

Database Scaling

- Read replicas for analytics queries
- Partition large tables by tenant_id

- Move large tenants to dedicated databases

CDN for Static Assets

- Serve frontend from CDN (CloudFront)
- Cache images, videos, PDFs

15. Disaster Recovery

Backup Strategy

- Daily automated database backups (7-day retention)
- S3 versioning enabled
- Point-in-time recovery (PITR)

Recovery Plan

- RTO (Recovery Time Objective): 4 hours
- RPO (Recovery Point Objective): 24 hours
- Automated failover to standby database